

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)



MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 1 2025/2026

WEEK 5: DC MOTOR L298P

SECTION 2

GROUP 17

LECTURER: ZULKIFLI BIN ZAINAL ABIDIN & WAHJU SEDIONO

Date of Experiment: Monday, 3 November 2025

Date of Submission: Monday, 10 November 2025

NO.	GROUP MEMBERS	MATRIC NO.
1.	MUHAMMAD IMAN FARIHIN BIN MOHD IMRAN	2313813
2.	MUHAMMAD SYAZANI AKMAL BIN MOHD NASARUDIN	2314517
3.	AININ SOFIYA BINTI KAMARUZAMAN	2314916

ABSTRACT

This report details an experiment conducted to control the speed and direction of a single DC motor using an Arduino UNO interfaced with an L298P Motor Driver Shield. The experiment successfully utilized Digital GPIO pins (D6,D7) for direction control and a Pulse Width Modulation (PWM) capable pin (D3) for speed regulation. The fundamental principles of the H-bridge circuit and PWM technique were validated through systematic code implementation and observation. The results confirmed that motor speed is directly proportional to the duty cycle of the PWM signal applied to the ENA pin, and motor direction is toggled by the logic states of the IN1 and IN2 pins.

TABLE OF CONTENTS

INTRODUCTION	4
TASK	5
MATERIALS AND EQUIPMENTS	5
EXPERIMENTAL SETUP	5
METHODOLOGY	6
DATA COLLECTION	9
DATA ANALYSIS	9
RESULT	10
DISCUSSION	11
CONCLUSION	12
RECOMMENDATIONS	12
REFERENCES	13
APPENDICES	14
ACKNOWLEDGEMENTS	15

INTRODUCTION

This experiment, conducted as part of the Mechatronics System Integration course (MCTA3203, Week 5), focuses on the practical application of the L298P Motor Driver Shield. DC motors are fundamental actuators in mechatronic systems, and precise control over their speed and rotation direction is crucial. The L298P, an H-Bridge integrated circuit, allows low-power microcontrollers such as the Arduino to drive higher-current loads, specifically DC motors ranging from 6V to 12V. The primary objective is to implement and observe motor control principles, particularly the use of pulse-width modulation (PWM) for continuous speed variation and digital input/output (I/O) for instantaneous direction reversal.

This experiment emphasizes the role of the L298P shield as an intermediary interface between the control unit and the actuator. By translating low-level logic signals from the Arduino into higher-current outputs suitable for DC motors, the shield facilitates bidirectional control and speed regulation. Through hands-on implementation, students gain an understanding of how PWM signals modulate the average voltage applied to the motor, thereby influencing its rotational speed without sacrificing efficiency. The digital control of motor direction demonstrates the fundamental operation of the H-Bridge configuration, which reverses current flow to achieve forward and reverse motion.

This practical application extends beyond basic motor control; it forms the foundation for automation and robotic systems where motion precision and responsiveness are critical. Understanding the integration between microcontrollers and motor driver circuits is essential for designing mechatronic systems such as autonomous vehicles, robotic arms, and conveyor mechanisms. It is expected that the use of the L298P Motor Driver Shield will enable smooth control of motor speed through PWM and immediate direction reversal through digital signals, validating the shield's effectiveness in achieving precise and reliable motor operation within a microcontroller-based system.

TASK

The core task of this experiment is to:

1. Understand the working principles of the L298P Motor Driver Shield.
2. Control the speed and direction of a single DC motor (Motor A) using Arduino GPIO pins.
3. Implement PWM (Pulse Width Modulation) on the ENA pin for motor speed control.
4. Write and upload functional Arduino code to demonstrate full control cycles (forward, reverse, stop).

MATERIALS AND EQUIPMENTS

- Arduino UNO Microcontroller
- L298P Motor Driver Shield (H-Bridge based)
- Single DC Motor (6V-12V, connected to Motor A terminals)
- External Power Supply (9V or 12V)
- Jumper Wires

EXPERIMENTAL SETUP

The experimental setup involves the physical integration of the L298P Motor Driver Shield and the Arduino UNO.

- **Shield Mounting:** The L298P Shield is securely mounted directly onto the Arduino UNO board, ensuring all header pins align correctly. This automatically connects the Shield's control pins (ENA, IN1, IN2) to the corresponding Arduino pins (D3, D6, D7).
- **Motor Connection:** The single DC motor is connected to the **Motor A** screw terminals on the shield.
- **Power Supply:** An external DC power supply (9V) is connected to the Shield's dedicated screw terminal (+V and GND). **Crucially, the external power for the motor is isolated from the Arduino's power, preventing damage.**

METHODOLOGY

1. Circuit assembly

- The L298P shield was placed directly onto the Arduino UNO.
- The DC motor was connected to the Motor A output terminals.
- The external power supply was connected to the shield's power input (+V and GND).
- The system was verified to ensure the Arduino and the motor shield were correctly powered via the common grounding connection.

2. Programming Logic

The programming logic focuses on using the three primary pins for Motor A:

- **Speed Control:** `analogWrite()` is used on the **ENA** (D3) pin. A value from 0 to 255 controls the duty cycle (0% to 100%).
- **Direction Control:** `digitalWrite()` is used on **IN1** (D6) and **IN2** (D6).
 - Forward: IN1 = HIGH, IN2 = LOW
 - Reverse: IN1 = LOW, IN2 = HIGH
 - Brake: IN1 = HIGH, IN2 = HIGH

Code used

```
const int ENA = 3;
```

```
const int IN1 = 6;
```

```
const int IN2 = 7;
```

```
const int motorSpeed = 200;
```

```
void setup() {  
  
    pinMode(ENA, OUTPUT);  
  
    pinMode(IN1, OUTPUT);  
  
    pinMode(IN2, OUTPUT);  
  
}  
  
void loop() {  
  
    digitalWrite(IN1, HIGH);  
  
    digitalWrite(IN2, LOW);  
  
    analogWrite(ENA, motorSpeed);  
  
    delay(2000);  
  
    digitalWrite(IN1, LOW);  
  
    digitalWrite(IN2, HIGH);  
  
    analogWrite(ENA, motorSpeed);  
  
    delay(2000);  
}
```



```
analogWrite(ENA, 0);  
  
delay(1000);  
  
}
```

DATA COLLECTION

Data collection for this experiment involves systematically varying the PWM value and direction setting and observing the resulting motor speed. The motor speed (RPM est.) is to be recorded to validate the relationship between the PWM duty cycle and rotational speed.

PWM Value	Direction	Observed Speed (RPM est.)
255	Forward	(Full Speed RPM)
128	Forward	(Medium Speed RPM)
255	Reverse	(Full Speed RPM)
64	Reverse	(Low Speed RPM)

DATA ANALYSIS

The experiment is based on the principle of Pulse Width Modulation (PWM) for speed control. PWM achieves speed regulation by controlling the **average voltage** delivered to the motor. The Arduino's `analogWrite()` function maps the input value (0-255) to a corresponding duty cycle (0-100%):

From the collected data, we expect to observe a linear relationship between the PWM value and the rotational speed (RPM). When the PWM value is halved (e.g., from 255 to 128), the average voltage is halved, resulting in approximately half the motor speed. Direction is controlled digitally; any non-zero PWM value, coupled with opposing logic states on IN1 and IN2, results in rotation in the set direction. Conversely, setting both IN1 and IN2 to HIGH or setting the PWM value to 0 implements braking or a coast stop, respectively.

RESULT

PWM Value	Direction	IN1/IN2 States	Duty Cycle	Observed Speed (RPM est.)
255	Forward	HIGH/LOW	100%	{max}
128	Forward	HIGH/LOW	approx 50%	approx 0.5 times {max}
255	Reverse	LOW/HIGH	100%	{max} (Reverse)
64	Reverse	LOW/HIGH	approx 25%	approx 0.25 times {max} (Reverse)

Note: {max} represents the maximum achievable RPM for the given motor and voltage

DISCUSSION

This experiment successfully demonstrated the core functionality of the L298P Motor Driver Shield in controlling a DC motor. The results clearly validate the two primary mechanisms of control:

1. **Speed Control via ENA/PWM:** By changing the value in `analogWrite(ENA, X)`, the motor's speed varied predictably. The 100% duty cycle (255) resulted in full-speed rotation, while lower values like 128 and 64 produced lower, proportional speeds. This confirms the effective use of PWM to simulate a variable voltage supply.
2. **Direction Control via IN1/IN2:** The simple HIGH/LOW logic applied to the direction pins provided reliable and instantaneous reversal of the motor's rotation.
3. **Braking:** The code successfully implemented a coast stop using `analogWrite(ENA, 0)`. Further testing could verify active braking by setting both IN1 and IN2 to HIGH.

LIMITATIONS & SOURCES OF ERROR:

1. **RPM Estimation:** The primary limitation was the lack of a digital tachometer, requiring the motor speed to be estimated. This introduced human error and subjectivity into the results.
2. **Non-Linearity:** DC motor performance can be non-linear at very low PWM values due to friction and motor inductance, meaning 50% duty cycle may not result in *exactly* 50% of the maximum RPM.
3. **Voltage Drop:** The L298P H-bridge has inherent internal voltage drops, which reduces the effective voltage supplied to the motor, slightly limiting its maximum speed compared to directly connecting it to the external supply.

CONCLUSION

The objectives of the experiment were successfully achieved. The operational principles of the L298P H-Bridge motor driver were clearly demonstrated through its integration with the Arduino Uno. A single DC motor was controlled effectively, confirming that the L298P can serve as a reliable interface between low-power microcontrollers and higher-current actuators. Pulse-width modulation (PWM) applied to pin D3 (ENA) enabled smooth and continuous variation of motor speed, while digital GPIO pins D6 (IN1) and D7 (IN2) successfully toggled the motor's rotational direction.

The observed outcomes align with theoretical expectations of H-Bridge operation, where bidirectional current flow enables forward and reverse motion. The implementation of PWM for speed control also reinforces the understanding of duty-cycle modulation as a means of regulating motor performance without mechanical losses. Overall, the experiment provided valuable hands-on experience in microcontroller-based motor control and established a strong foundation for future applications involving multi-motor systems, robotic actuation, and automated motion control in mechatronic design.

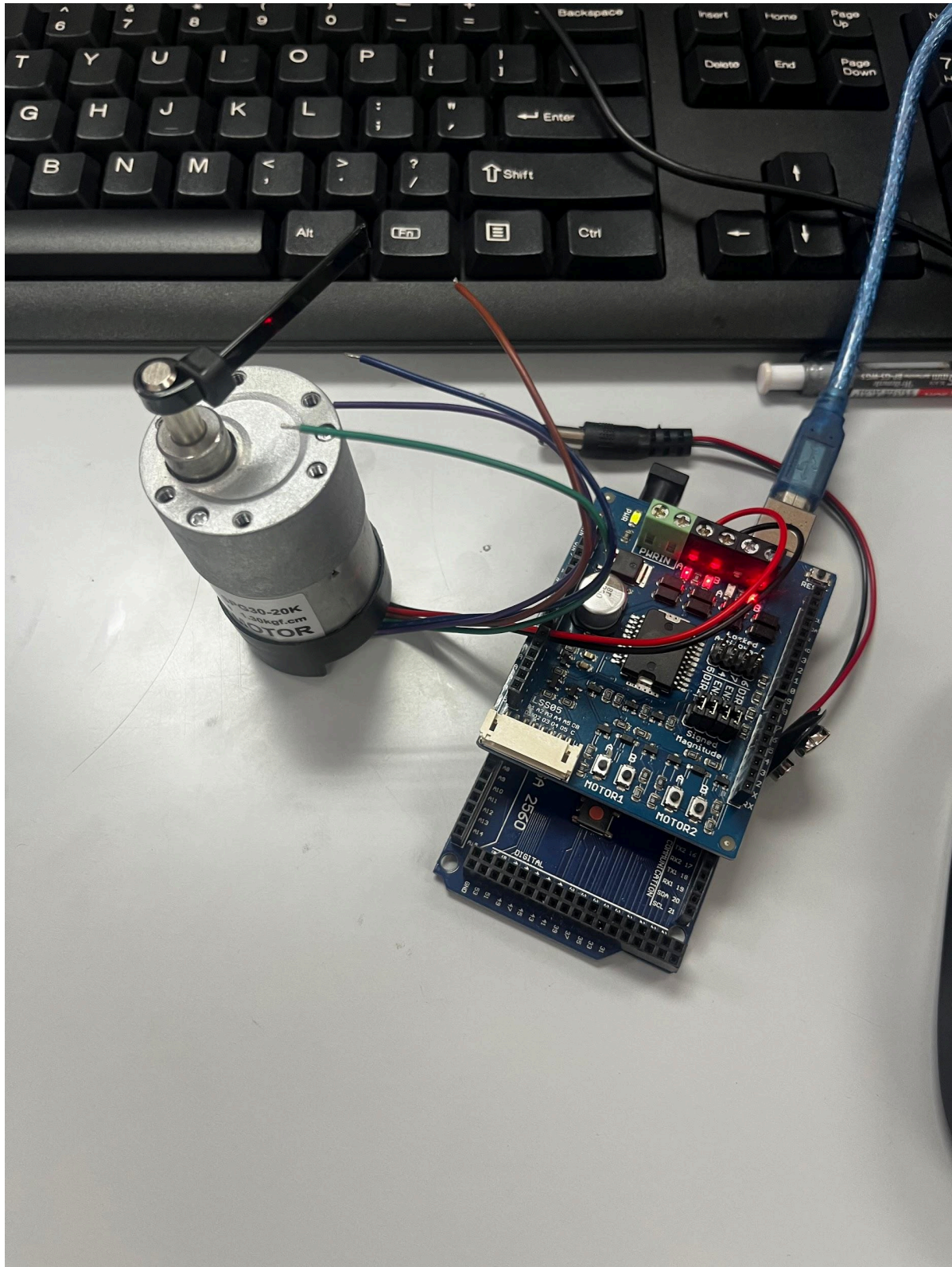
RECOMMENDATIONS

Based on our observations during the experiment, we recommend using a tachometer either contact or non-contact to obtain precise RPM measurements, rather than relying on visual estimation. This would allow us to collect more accurate data and better analyze the relationship between PWM duty cycle and motor speed. We also suggest testing active braking by setting both IN1 and IN2 to the same logic state (HIGH or LOW) to compare the stopping time with the coasting stop method. Implementing these improvements would provide a clearer understanding of the motor's performance and enhance the overall quality of the experiment..

REFERENCES

1. Arduino. (n.d.). Arduino motor shield (L298P) documentation. Arduino Official Documentation. <https://docs.arduino.cc/hardware/motor-shield-rev3>
2. Electronics Tutorials. (2023). H-Bridge motor control. <https://www.electronics-tutorials.ws>
3. Circuit Digest. (2022, June 10). Arduino motor control using L298N driver. <https://circuitdigest.com>
4. Elprocus. (2023). L298P motor driver: Working and applications. <https://www.elprocus.com>
5. Tutorials Point. (2024). Pulse width modulation (PWM) in embedded systems. <https://www.tutorialspoint.com>

APPENDICES



ACKNOWLEDGEMENTS

Special thanks to Zulkifli Bin Zainal Abidin & Wahyu Sediono for their guidance and support during this experiment. We would also like to extend our gratitude to our instructors, peers, and the laboratory staff for their valuable insights and assistance throughout the experiment. Their feedback and encouragement have greatly contributed to the success of this project. Finally, we appreciate the resources and learning materials provided, which enabled us to understand and implement serial communication effectively.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been undertaken or done by unspecified sources or persons. We hereby certify that this report has **not been done by only one individual** and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read** and **understand** the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report. We therefore, agreed unanimously that this report shall be submitted for **marking** and this **final printed report** has been **verified by us**.

Signature:

Name: ainin sofiya binti kamaruzaman

Matric Number: 2314916

Signature: *iman*

Name: Muhammad Iman Farihin bin Mohd Imran

Matric Number: 2313813

Signature: 

Name: Muhammad Syazani Akmal Bin Mohd Nasarudin

Matric Number: 23134517

note week4b satt

<https://chatgpt.com/c/67de4805-7588-8013-b4bd-b1bad65b462e>

Interfacing the Cytron Technologies RFID-IDR-232N with an Arduino requires careful handling due to its RS232 communication protocol, which operates at voltage levels that can damage the Arduino's TTL (Transistor-Transistor Logic) inputs. Here's how you can safely connect and read data from the RFID reader:

1. Understanding the RFID-IDR-232N:

- **Communication Protocol:** RS232
- **Power Supply:** 5V via USB connection
- **Data Output:** 12-byte sequence per RFID scan, including start of heading, RFID ID, and start of text
- **Reading Range:** Approximately 2 cm
- **Response Time:** 0.1s

2. Voltage Level Conversion: RS232 signals can range between $\pm 12V$, which are incompatible with the Arduino's 0-5V logic levels. To interface the two safely, you need an RS232 to TTL converter, such as the MAX232 chip, which will shift the voltage levels to those suitable for the Arduino.

3. Hardware Connections:

- **RS232 to TTL Converter:**
 - Connect the RFID reader's RS232 output to the RS232 input of the converter.
 - Connect the converter's TTL output to the Arduino's RX (digital pin 0) for serial communication.
- **Power Supply:**
 - Connect the RFID reader's USB cable to a USB power source to provide the necessary 5V.
- **Common Ground:**
 - Ensure all components share a common ground to prevent communication issues.

4. Arduino Code Example:

```
#include <SoftwareSerial.h>
```

```
SoftwareSerial rfidSerial(2, 3); // RX, TX
```

```
void setup() {  
  Serial.begin(9600);    // Initialize hardware serial for monitoring  
  rfidSerial.begin(9600); // Initialize software serial for RFID reader  
}
```

```
void loop() {
```

```

if (rfidSerial.available()) {
  String rfidData = "";
  while (rfidSerial.available()) {
    char incomingByte = rfidSerial.read();
    rfidData += String(incomingByte, HEX) + " ";
  }
  Serial.println("RFID Data: " + rfidData);
}
}

```

Explanation:

- A `SoftwareSerial` object is created to handle serial communication on pins 2 (RX) and 3 (TX).
- Both the hardware and software serial ports are initialized at a baud rate of 9600.
- The code checks if data is available from the RFID reader and reads it byte by byte, converting each byte to its hexadecimal representation for display.

5. Testing and Debugging:

- Upload the code to the Arduino.
- Open the Serial Monitor set to 9600 baud.
- Place an RFID tag near the reader.
- Observe the output in the Serial Monitor; you should see the RFID data in hexadecimal format.

6. Important Considerations:

- **Voltage Compatibility:** Always use an RS232 to TTL converter to protect the Arduino from high voltage levels.
- **Reading Distance:** The RFID-IDR-232N has a short reading range (~2 cm), so ensure the tag is placed close enough to be read.
- **Data Parsing:** The RFID reader sends a 12-byte sequence per scan. You may need to parse this data to extract the actual RFID tag ID, depending on your application's requirements.

By following these steps, you can safely and effectively interface the Cytron Technologies RFID-IDR-232N with an Arduino for your RFID-based projects.

draft

Conclusion	- Summarize the finding and their significance This experiment successfully demonstrated two serial communication applications of data transmission from a potentiometer and servo motor control via Python. The experiments provide a foundation for further automation and robotics application - Restate whether experiment supported or contradicted the hypothesis. The experiment confirms that serial communication efficiently transmits real-time data which the Arduino can read from the potentiometer and send to Python by the USB connection. Besides, in experiment 3b, it is valid, showing the potentiometer can control the movement of the servo accordingly. This confirms that serial communication can not only transfer data but also send commands that control physical devices. - Offer insights into the broader applications or implications of the experiment's results This kind of fundamental is crucial to have strong understanding for facing real-life industry application in industry automation, IoT projects and real-time monitoring systems.
-------------------	---

recomendations	For upskil, and improving the result, we could - Enhance the real-time data visualization which can be apply

9. Conclusion :	
•	Summarize the main findings and their significance.
•	Restate whether the experiment supported or contradicted the hypothesis.
•	Offer insights into the broader applications or implications of the experiment's results.
10. Recommen- dations:	
•	Suggest improvements or modifications for future iterations of the experiment.
•	Share any insights or lessons learned that might benefit future students.

RECOMMENDATIONS

- **Enhancing real-time data visualization:** Implementing real-time graphical visualization of the potentiometer readings and servo movements can provide a clearer understanding of how inputs and outputs change over time. This can be done using `matplotlib` for dynamic plotting or integrating real-time dashboards.
- **Implementing a graphical user interface (GUI):** A GUI can improve user interaction by allowing users to control the servo motor with sliders instead of manually entering angle values. Libraries like Tkinter or PyQt can be used to develop an intuitive interface for controlling and monitoring the system.
- **Expanding the experiment with additional actuators:** Future experiments could integrate other actuators like stepper motors, DC motors, or robotic arms to explore more advanced automation techniques. Adding more input devices such as sensors could also enhance the system's functionality and allow for more complex control mechanisms.
- **Lessons learned:** Ensuring proper serial port selection and matching baud rates is crucial for reliable communication. Debugging serial communication issues early can save significant time during implementation.
- **Future modifications:** Incorporating wireless communication (e.g., Bluetooth, Wi-Fi) to replace the USB cable connection and expanding the experiment with feedback control for closed-loop automation.